МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО”

Факультет прикладної математики
Кафедра програмного забезпечення комп’ютерних систем

Лабораторна робота № 3

з дисципліни “ Прикладне програмне забезпечення мовою Python”
тема “Робота з файлами та форматування даних у формат JSON”

Виконав
Студент III курсу
групи КП-31
Власенко Артем

Київ 2026

Варіант 2

1. Написати функцію, яка приймає шлях до файлу та кількість рядків, які потрібно прочитати та записує ці рядки в файл.

2. Написати функцію, приймаючу словник, який може містити будь-які елементи та конвертує у *JSON* тільки ті елементи, значення яких є списками, що містять лише цілочислові значення.

3. Написати функцію, яка приймає шлях до *HTML* файлу та *html* тег ("*p*", "*h1*", "*article*" та інше) і повертає кількість повторень отриманого тега у файлі з урахуванням того, що потрібно повернути тільки кількість тегів, що мають відкриваючу та закриваючу частину.

4. У двох файлах записані відсортовані за зростанням масиви невідомої довжини. Написати функцію, яка об'єднує їх та записує результат у новий файл. Отриманий масив також має бути відсортований за зростанням.

5. Напишіть функцію mk_pkg(path, root_path='./'), що створює по надісланому у вигляді рядка відносному шляху path готовий python-пакет (тобто директорія із файлом ініціалізації __init__.py). У разі відсутності будь-яких проміжних пакетів або наявності директорій без файлів ініціалізації, функція повинна створювати їх. Необов'язковий аргумент root_path='./' повинен приймати рядок з директоріями, щодо якого буде вираховуватись місце розташування цільової директорії (за замовчанням як локальний корінь повинен використовуватися директорія скрипта). Коли функція буде готова, перевірте її, попередньо створивши в

каталозі скрипта звичайний директорія pkg_1, а потім здійснивши два виклики функції: mk_pkg('pkg_1/pkg_2') та mk_pkg('pkg_1/pkg_3/pkg_4').

```python
import json
import heapq
from pathlib import Path
from bs4 import BeautifulSoup


# Task 1: Read N lines from a file and write them to another file
def read_and_write_lines(src_path: str, n: int, dst_path: str) -> None:
    with open(src_path, 'r') as src:
        lines = [src.readline() for _ in range(n)]
    with open(dst_path, 'w') as dst:
        dst.writelines(lines)


# Task 2: Convert to JSON only dict entries whose values are lists of ints
def dict_to_json(data: dict) -> str:
    filtered = {
        k: v for k, v in data.items()
        if isinstance(v, list) and v and all(isinstance(x, int) for x in v)
    }
    return json.dumps(filtered)


# Task 3: Count non-self-closing occurrences of an HTML tag in a file
def count_html_tags(html_path: str, tag: str) -> int:
    with open(html_path, 'r', encoding='utf-8') as f:
        soup = BeautifulSoup(f, 'html.parser')
    return sum(1 for t in soup.find_all(tag) if f'</{tag}' in str(t))
```

```python
# Task 4: Merge two sorted files into a new sorted file
def merge_sorted_files(path1: str, path2: str, dst_path: str) -> None:
    def read_ints(path):
        with open(path, 'r') as f:
            return [int(line.strip()) for line in f if line.strip()]

    merged = list(heapq.merge(read_ints(path1), read_ints(path2)))
    with open(dst_path, 'w') as out:
        out.write('\n'.join(map(str, merged)) + '\n')


# Task 5: Create a Python package hierarchy, adding __init__.py where missing
def mk_pkg(path: str, root_path: str = './') -> None:
    base = Path(root_path).resolve()
    parts = Path(path).parts

    current = base
    for part in parts:
        current = current / part
        current.mkdir(exist_ok=True)
        init = current / '__init__.py'
        if not init.exists():
            init.touch()
```

```python
if __name__ == '__main__':
    import os
    import tempfile

    # Task 1
    with tempfile.NamedTemporaryFile('w', delete=False, suffix='.txt') as tmp_src:
        tmp_src.write('line one\nline two\nline three\nline four\n')
        src_name = tmp_src.name
    dst_name = src_name + '_out.txt'
    read_and_write_lines(src_name, 2, dst_name)
    with open(dst_name) as f:
        print(f.read(), end='')

    # Output:
    # line one
    # line two

    # Cleanup
    os.unlink(src_name)
    os.unlink(dst_name)


    # Task 2
    print(dict_to_json({'a': [1, 2, 3], 'b': ['x', 'y'], 'c': [], 'd': [4, 5]}))
    # {"a": [1, 2, 3], "d": [4, 5]}
    print(dict_to_json({'x': 'hello', 'y': [1, 'two'], 'z': [7]}))
    # {"z": [7]}
```

```python
# Task 3
with tempfile.NamedTemporaryFile('w', delete=False, suffix='.html') as tmp_html:
    tmp_html.write('<html><body><p>One</p><p>Two</p><br/><p>Three</p></body></html>')
    html_name = tmp_html.name

print(count_html_tags(html_name, 'p'))   # 3
print(count_html_tags(html_name, 'br'))  # 0  (<br/> is self-closing)

# Cleanup
os.unlink(html_name)


# Task 4
with tempfile.NamedTemporaryFile('w', delete=False, suffix='.txt') as f1, \
     tempfile.NamedTemporaryFile('w', delete=False, suffix='.txt') as f2:
    f1.write('1\n3\n5\n7\n')
    f2.write('2\n4\n6\n')
    p1, p2 = f1.name, f2.name
merged_name = p1 + '_merged.txt'

merge_sorted_files(p1, p2, merged_name)
with open(merged_name) as f:
    print(f.read(), end='')

# Output:
# 0
# 1
# 2
# 3
# 4
# 5
# 6
# 7

# Cleanup
os.unlink(p1)
os.unlink(p2)
os.unlink(merged_name)
```

```python
# Task 5
import shutil
test_root = Path('./test_pkg_root')
test_root.mkdir(exist_ok=True)
(test_root / 'pkg_1').mkdir(exist_ok=True)

mk_pkg('pkg_1/pkg_2', root_path=str(test_root))
mk_pkg('pkg_1/pkg_3/pkg_4', root_path=str(test_root))

for p in sorted(test_root.rglob('*')):
    print(p.relative_to(test_root))

# Output:
# pkg_1s
# pkg_1/__init__.py
# pkg_1/pkg_2
# pkg_1/pkg_2/__init__.py
# pkg_1/pkg_3
# pkg_1/pkg_3/__init__.py
# pkg_1/pkg_3/pkg_4
# pkg_1/pkg_3/pkg_4/__init__.py

# Cleanup
shutil.rmtree(test_root)
```

Висновки

Виконуючи дану лабораторну роботу я ознайомився з основними способами роботи з файлами в Python, зокрема з використанням функції open() та конструкції with для безпечного відкриття і закриття файлів. Я навчився виконувати синтаксичний аналіз HTML-документів за допомогою бібліотеки BeautifulSoup, а також опанував роботу з форматом JSON через вбудований модуль json для серіалізації та десеріалізації даних. Отримані навички дозволяють ефективно обробляти як локальні файли різних форматів, так і структуровані дані з веб-ресурсів. Таким чином, робота з файлами та форматом JSON є важливою складовою розробки сучасних програм, що взаємодіють із зовнішніми джерелами даних.

Питання для самоперевірки

1. Файл — це набір даних, збережених на носії під певним ім'ям. Розширення вказує на тип і формат файлу, що допомагає системі та програмам коректно його обробляти.
2. Директорії дозволяють організовувати файли у логічні групи та спрощують навігацію файловою системою.
3. Файловий об'єкт — це програмне представлення з'єднання з файлом на диску, через яке виконується читання та запис даних
4. Текстові файлові об'єкти — для роботи з текстом, бінарні — для роботи з нетекстовими даними (зображення, аудіо тощо).
5. Викликати open(path, 'r', encoding='utf-8') і застосувати метод read() або readlines() до отриманого об'єкта.
6. Викликати open(path, 'a', encoding='utf-8') і записати рядок методом write() — режим 'a' дописує дані в кінець, не очищуючи файл.
7. Передати відкритий файловий об'єкт у параметр file функції print(): print('текст', file=f).
8. Текстові файли зберігають людиночитабельні дані та відкриваються у текстовому редакторі. Бінарні файли зберігають дані у двійковому форматі й не призначені для прямого читання як текст.
9. Закриття файлу звільняє системні ресурси та гарантує запис буферизованих даних на диск.
10. Серіалізація — це перетворення об'єкта у формат, придатний для збереження або передачі, наприклад у файл чи через мережу.
11. Модуль pickle для довільних Python-об'єктів, модуль json — для серіалізації у формат JSON.
12. Так, файлові об'єкти є контекстними менеджерами — конструкція with автоматично закриває файл після виходу з блоку.

13. JSON (JavaScript Object Notation) — текстовий формат обміну даними, що підтримує представлення об'єктів у вигляді пар ключ-значення та масивів.
14. За допомогою модуля json: json.loads() для рядка або json.load() для файлового об'єкта.
15. Відкрити файл через open(path, 'r', encoding='utf-8') і передати його в json.load(), що коректно обробить вкладені структури будь-якої складності.